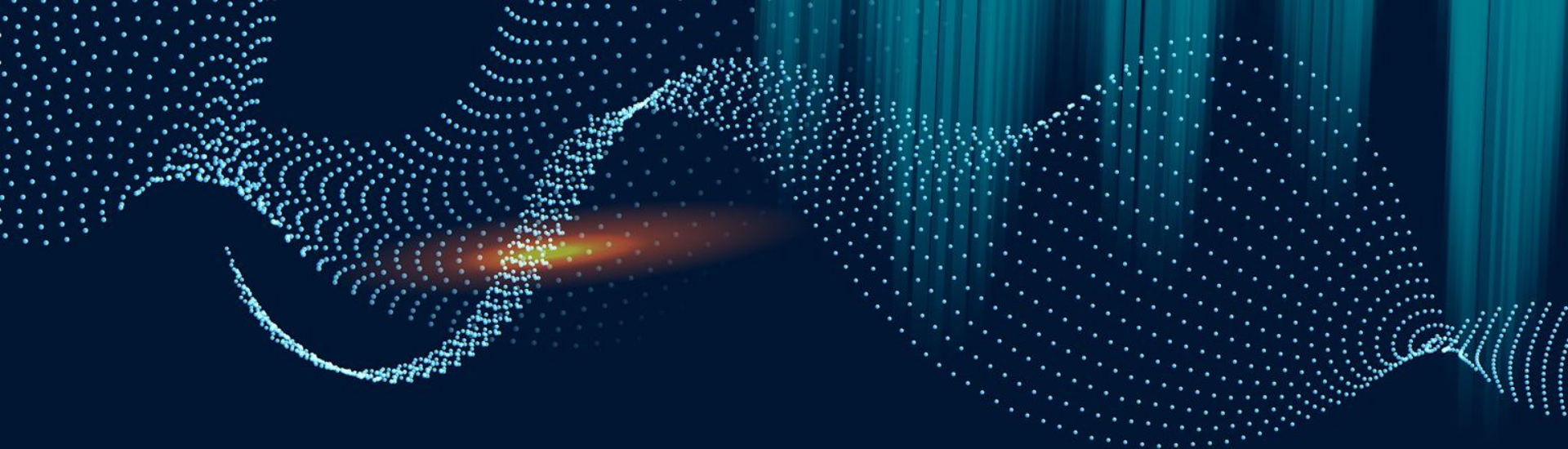


Fundamentals of Deep Learning, Network Design, and Training

Dr. Pejman Ebrahimi

Deep Learning and Advanced AI Techniques Course



Neural Networks

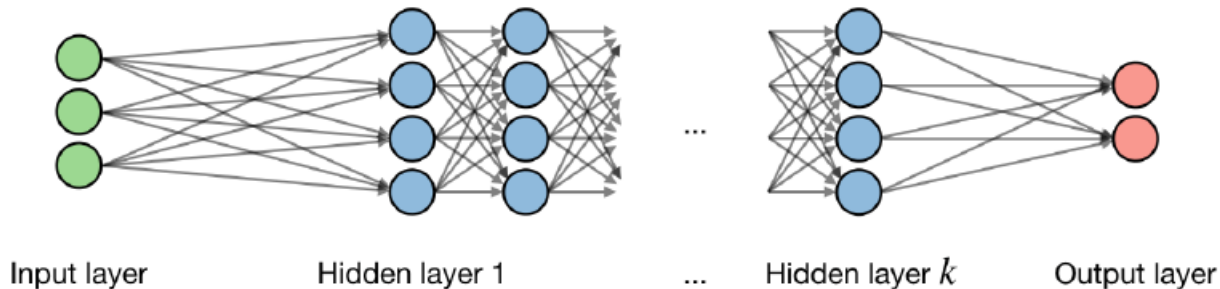
Neural Network - Definition

A neural network is a kind of computer program that's designed to think a little like a human brain. It's made up of layers of tiny units called **neurons**, and each neuron processes information and passes it along to the next layer.

Here's a simple way to think about it:

- **Input Layer:** Takes in the raw information (like an image, numbers, or words).
- **Hidden Layers:** Do lots of calculations behind the scenes. These layers find patterns or important features in the information.
- **Output Layer:** Gives the final result (like saying "this is a cat" or "this is not spam").

Neural Network - Architecture



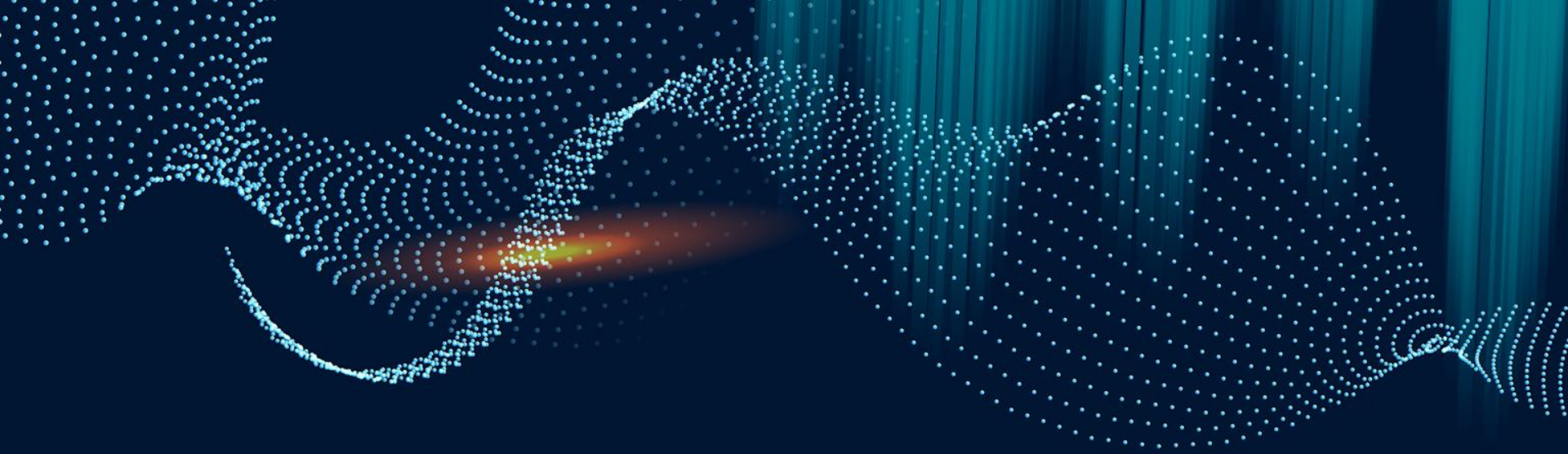
By noting i the i^{th} layer of the network and j the j^{th} hidden unit of the layer, we have:

$$z_j^{[i]} = w_j^{[i]T} x + b_j^{[i]}$$

where we note w , b , z the weight, bias and output respectively.

where we note w , b , z the weight, bias and output respectively.

$$z_j^{[i]} = w_j^{[i]T} x + b_j^{[i]}$$



Activation **Function**



Activation function

An **activation function** is a small but super important part of a neural network.

👉 Think of it like a decision-maker inside each neuron.

After a neuron adds up its input, the activation function decides **what should happen next?** should the neuron "fire" (send a signal) or stay quiet?

It adds **non-linearity** to the network — meaning the network can learn complex stuff like recognizing faces, understanding speech, or playing games. Without activation functions, a neural network would just be a boring calculator that can only handle simple problems

ReLU (Rectified Linear Unit)

🧠 Idea:

If you give it a **positive number**, it keeps it.

If you give it a **negative number**, it throws it away and gives back zero.

✏️ Formula:

$$f(x) = \max(0, x)$$

(That just means: choose the bigger one between 0 and x.)

✅ Example:

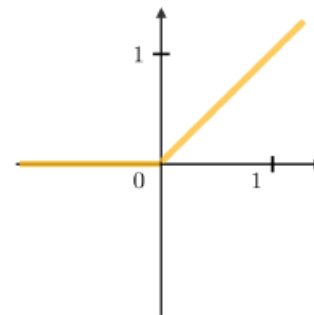
- Input = 5 → Output = 5
- Input = -3 → Output = 0
- Input = 0 → Output = 0

🔗 Why use it?

It's super fast and works really well for most problems, like image recognition.

ReLU

$$g(z) = \max(0, z)$$



Leaky ReLU

- ❑ Fixes the "dying ReLU" problem.
- ❑ Helps the network keep learning, even when some inputs are negative.
- ❑ Still simple and fast.

Instead of giving exactly 0 for negative inputs, Leaky ReLU gives a small, tiny negative output.

✓ Formula:

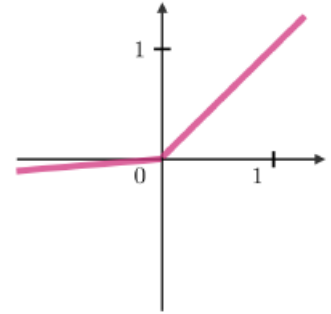
$$f(x) = \begin{cases} x & \text{if } x > 0 \\ 0.01x & \text{if } x \leq 0 \end{cases}$$

In simple words:

- If x is positive $\rightarrow$ keep it as it is.
- If x is negative $\rightarrow$ multiply it by a small number (like 0.01) instead of throwing it away completely.

Leaky ReLU

$$g(z) = \max(\epsilon z, z) \\ \text{with } \epsilon \ll 1$$



Dying ReLU Problem

When we use ReLU activation, remember:

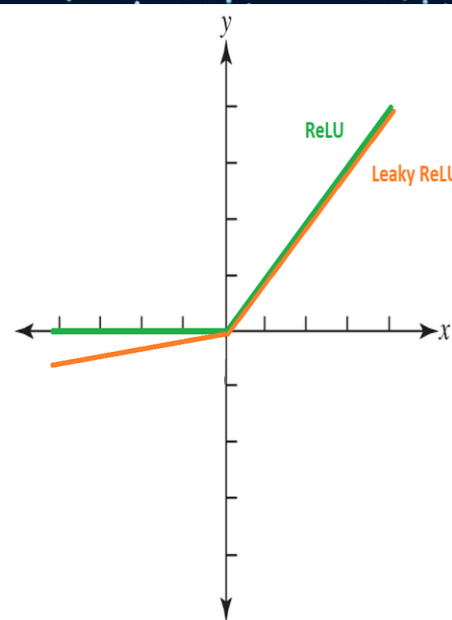
- If input $> 0 \rightarrow$ ReLU gives that number.
- If input $< 0 \rightarrow$ ReLU gives 0.

Now, imagine a situation during training:

- 👉 If a neuron keeps getting negative inputs again and again,
- 👉 It will always output 0,
- 👉 And it stops learning anything.

This neuron is dead — it's not helping the network anymore.

That's called the "Dying ReLU problem."



That's called the "Dying ReLU problem."

This is the Dying ReLU problem. It happens when the network is trained and the neuron always outputs 0.

© Dr. Pejman Ebrahimi

Sigmoid

🧠 Idea:

No matter what number you give it (big or small), it **squashes** the output **between 0 and 1**. It's great when you want a **probability** (because probabilities are always between 0 and 1).

✅ Example:

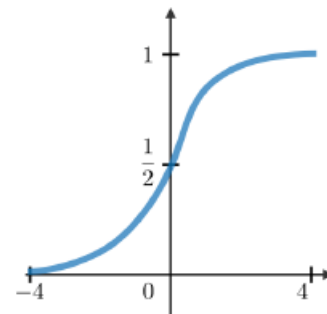
- Input = 100 → Output ≈ 1
- Input = 0 → Output = 0.5
- Input = -100 → Output ≈ 0

🎯 Why use it?

Good for problems where you're asking **yes/no** questions — like "Is this email spam?"

Sigmoid

$$g(z) = \frac{1}{1 + e^{-z}}$$



Tanh (short for hyperbolic tangent)

🧠 Idea:

Kind of like Sigmoid, but it squashes numbers between -1 and 1 instead of 0 and 1.

✅ Example:

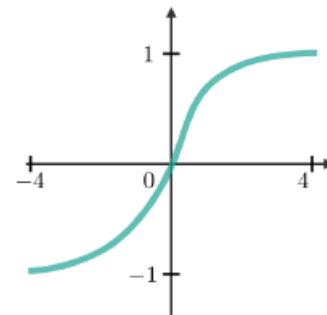
- Input = 100 → Output ≈ 1
- Input = 0 → Output = 0
- Input = -100 → Output ≈ -1

🔗 Why use it?

Better when you want **both** positive and negative outputs (like "how strongly positive" or "how strongly negative").

Tanh

$$g(z) = \frac{e^z - e^{-z}}{e^z + e^{-z}}$$



Softmax

✓ Softmax is usually used in the last layer when you want to do **classification** (choose between several categories).

✓ It turns a list of numbers into **probabilities** that add up to 1.

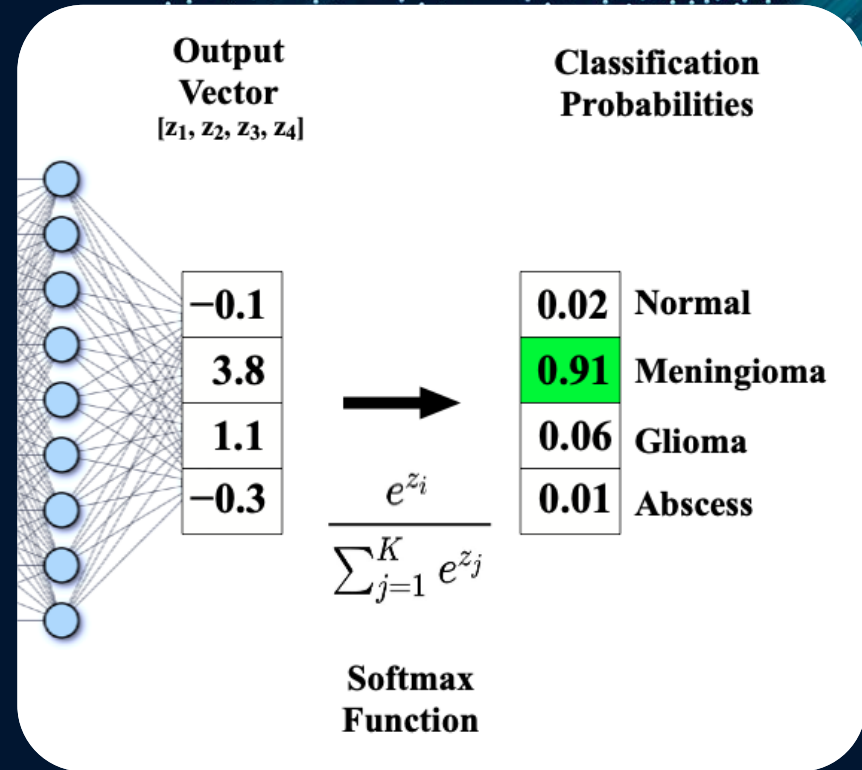
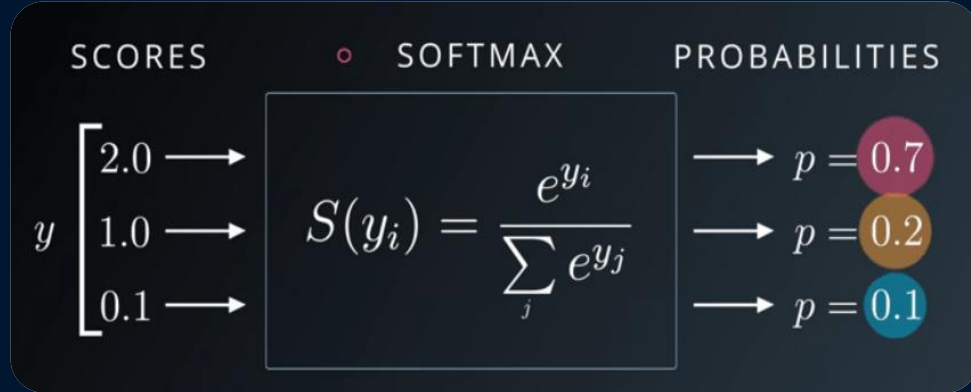
🧠 **Simple idea:**

- If you give it numbers like `[2, 1, 0.1]`, it will turn them into something like `[0.65, 0.25, 0.1]`.
- Bigger numbers become bigger probabilities.

• bigger numbers become bigger probabilities

• if you give it numbers like `[2, 1, 0.1]` it will turn them into something like `[0.65, 0.25, 0.1]`

Softmax



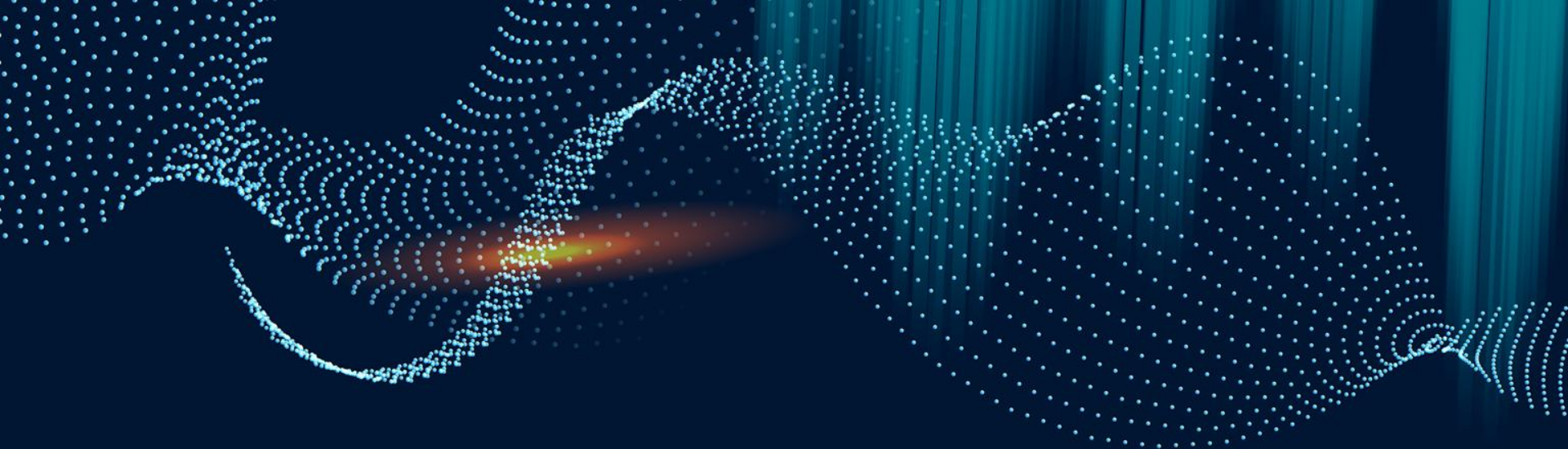
Activation function - Summary

Activation	Used For
ReLU	Hidden layers, fast and simple learning; but can have "dying ReLU" problem
Leaky ReLU	Hidden layers; fixes dying ReLU by allowing small negative outputs
Sigmoid	Output layer for binary classification (yes/no problems), like "cat or not"
Tanh	Hidden layers when you want outputs between -1 and 1; better than Sigmoid for zero-centered outputs
Softmax	Final output layer for multi-class classification (choosing between multiple categories)

Exercise - 1

Fill in the missing outputs based on the given input and the activation function

Input	ReLU Output	Leaky ReLU Output (slope=0.01)	Sigmoid Output	Tanh Output	Softmax Output (after all inputs)
2	?	?	?	?	?
-3	?	?	?	?	?
0	?	?	?	?	?
5	?	?	?	?	?



Loss Functions

Common Loss Functions

Loss Functions in Deep Learning

Purpose of Loss Functions

- Convert errors into a single scalar value that can be minimized
- Provide the gradient signal for backpropagation
- Define what "good performance" means for a specific task

Loss functions (also called cost or objective functions) quantify the error between predicted values (**y_pred**) and actual target values (**y_true**). They provide a measure of how well the model is performing and guide the optimization process during training.

Cross-Entropy Loss

Cross-Entropy is mainly used for: Classification problems (e.g., cat vs dog, digit 0-9, etc.)

- If your model predicts the right answer with high confidence → **small loss** ✓
- If your model predicts wrong or unsure → **big loss** ✗

It punishes wrong answers more heavily than some other loss functions.



If you predicted probability p for the correct class:

$$\text{Loss} = -\log(p)$$

- If $p = 1$ (perfect) → loss = 0
- If p is small → loss is big!

Common Loss Functions

Loss	Use case	Idea
Cross-Entropy Loss	Classification (2 or more classes)	Punishes wrong confident predictions heavily
Mean Squared Error (MSE)	Regression (predicting numbers)	Measures the average squared difference between actual and predicted
Mean Absolute Error (MAE)	Regression	Measures the average absolute difference (less sensitive to outliers than MSE)
Hinge Loss	Classification (especially for SVMs)	Helps in pushing correct class scores higher than wrong ones
Huber Loss	Regression	Mix between MSE and MAE, good when you want to handle outliers nicely

Categorical Cross-Entropy Loss

- ❑ Used for multi-class classification tasks (3 or more classes).
- ❑ Measures how far the model's predicted probabilities are from the true label.
- ❑ Lower loss means better predictions

$$\text{Loss} = - \sum (\text{true label} \times \log(\text{predicted probability}))$$

- Focuses only on the probability assigned to the correct class.
- Works well with **Softmax activation** in the final layer.

- Works well with softmax activation in the final layer.

Binary vs. Multi-class Classification

Task	Activation Function	Loss Function
Binary Classification (2 classes)	Sigmoid	Binary Cross-Entropy
Multi-class Classification (3 or more classes)	Softmax	Categorical Cross-Entropy

Binary
Classification



- Spam
- Not spam

Multiclass
Classification



- Dog
- Cat
- Horse
- Fish
- Bird
- ...

Multi-label
Classification

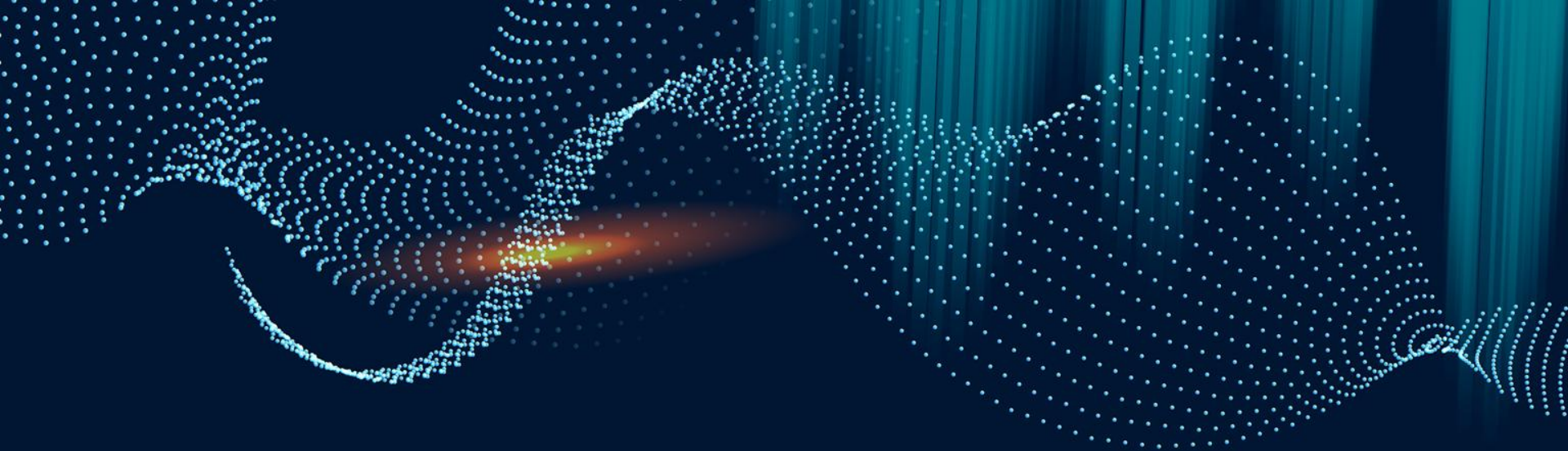


- Dog
- Cat
- Horse
- Fish
- Bird
- ...



Quick Quiz – 1 ⚡

- ❑ If a model predicts 99% correct class probability, cross-entropy loss will be (small / large)?
- ❑ If a model predicts 1% correct class probability, cross-entropy loss will be (small / large)?
- ❑ Which loss is mainly used in regression: Cross-Entropy, or Mean Squared Error?



Optimizer

Definition

An optimizer is an algorithm or method used to update the weights of a neural network during training to minimize the loss function.

It helps the model learn by adjusting parameters (weights and biases) iteratively based on the gradients of the loss function with respect to these parameters.

The goal is to find the optimal set of weights that minimizes the error between predicted and actual outputs.

Types of Optimizers

OPTIMIZER	ADVANTAGES	DISADVANTAGES
SGD	Simple, widely used	Slow convergence, sensitive to learning rate
Momentum	Faster convergence, reduces oscillations	Requires tuning of momentum parameter
Adam	Adaptive learning rates, fast convergence	May generalize poorly in some cases
RMSProp	Prevents vanishing learning rates	Sensitive to hyperparameters
AdaGrad	Handles sparse data well	Learning rate can become too small

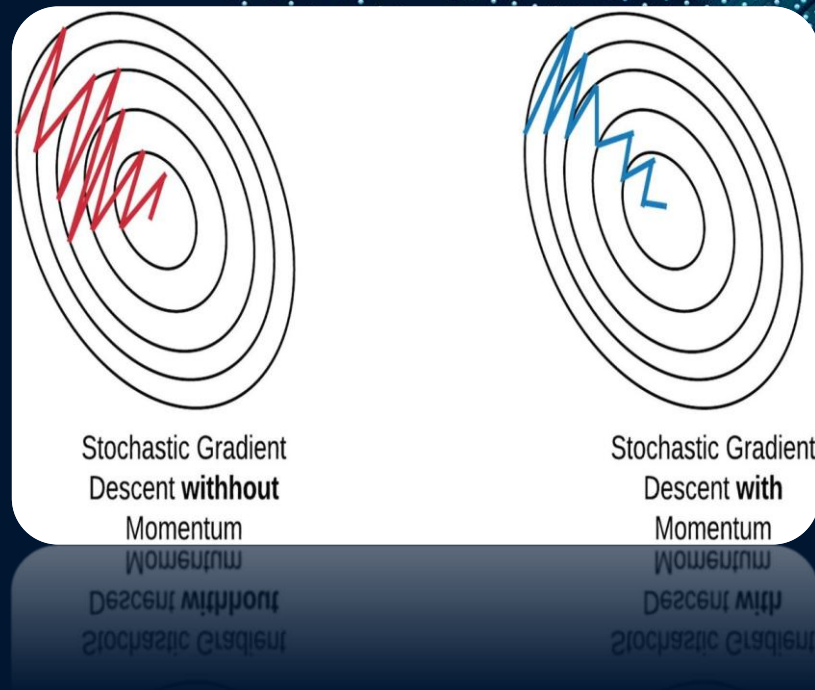
❑ **Learning rate** – The learning rate, often noted η , indicates at which pace the weights get updated. This can be fixed or adaptively changed. The current most popular method is called Adam, which is a method that adapts the learning rate.

Stochastic Gradient Descent (SGD)

- ❑ Stochastic = random.
- ❑ Learn step-by-step, one random sample at a time
- ❑ It uses random samples instead of the whole dataset each time.
- ❑ Makes learning faster and more dynamic, but a bit noisy.
- ❑ Updates weights using the gradient of the loss function with respect to the weights

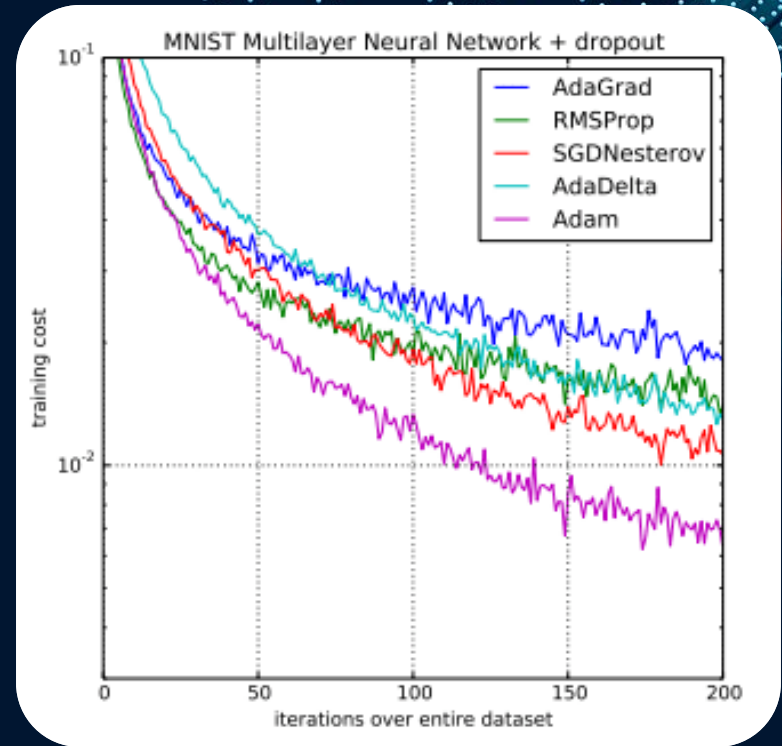
Momentum

- ❑ Improves SGD by helping it move faster in the right direction.
- ❑ It remembers the past gradients to keep moving smoothly, even if some steps are noisy.
- ❑ SGD alone can be slow and shaky (random movements).
- ❑ Momentum helps accelerate learning and reduces oscillations.



Adam (Adaptive Moment Estimation)

- Adam = Momentum + Adaptive Learning Rates
- It adjusts how much each weight should change, based on past gradients and past updates.
- Fast convergence (learns quickly).
- Works well even without much tuning.
- Best default choice for most deep learning tasks.
- Adam = Smart optimizer that moves fast and adapts to each weight**



RMSProp (Root Mean Square Propagation)

- ▣ Divides the learning rate by an exponentially decaying average of squared gradients.
- ▣ Prevents the learning rate from becoming too small over time.

The key formula for RMSprop is:

$$S_{dx} = \beta \times S_{dx} + (1 - \beta) \times (\nabla J)^2$$

$$\theta = \theta - \frac{\alpha}{\sqrt{S_{dx} + \epsilon}} \times \nabla J$$

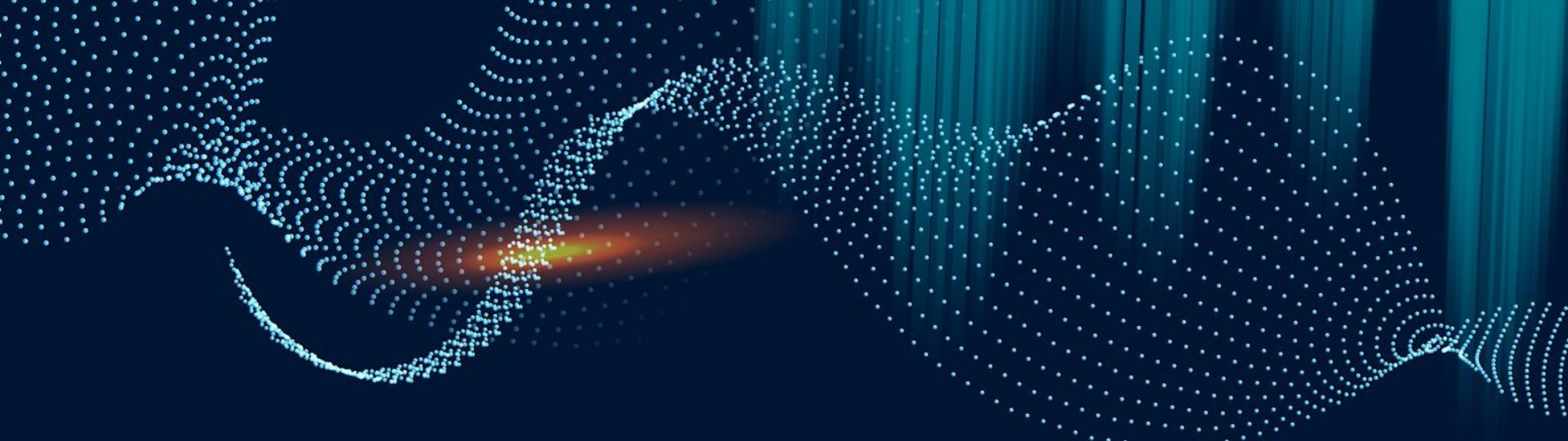
Where:

- S_{dx} is a running average of the squared gradient for parameter θ .
- β is a decay factor, typically set close to 1 (e.g., 0.9).
- ∇J is the gradient of the loss function J with respect to parameter θ .
- α is the global learning rate.
- ϵ is a small constant to prevent division by zero (e.g., $1e - 10$).



Quick Quiz – 2 ⚡

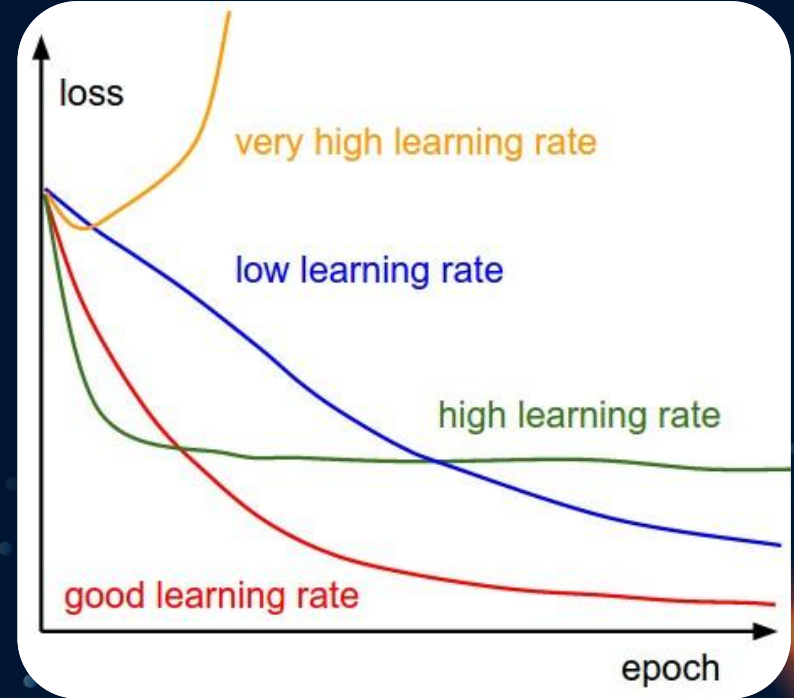
- ❑ Which optimizer converges the fastest?
- ❑ How does the choice of optimizer affect the final accuracy of the model?
- ❑ Experiment with different learning rates for each optimizer. What do you observe?

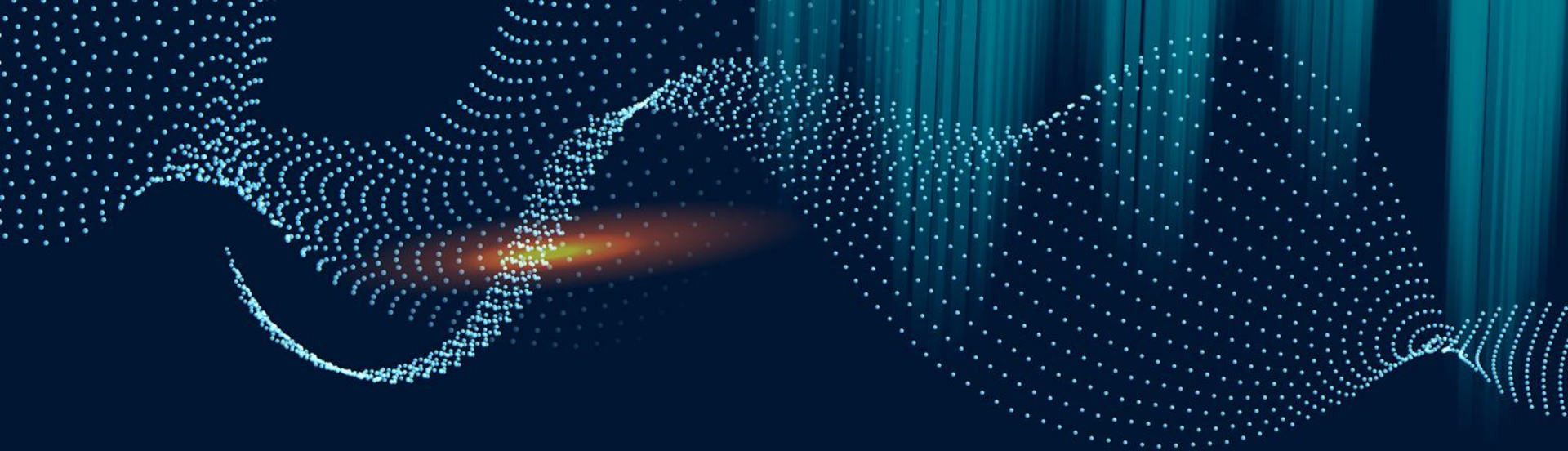


Learning **Rate**

Tips on learning rate

- ❑ Learning Rate controls how big the weight updates are during training.
- ❑ It decides how fast or slow the model learns.
- ❑ If learning rate is too high → model might jump around and never converge.
- ❑ If learning rate is too low → model will learn very slowly and might get stuck.
- ❑ For Adam → Start with 0.001
- ❑ For SGD → Start with 0.01
- ❑ If stuck → Try lowering by 10× (example: 0.001 → 0.0001)





Backpropagation

How Backpropagation Works?



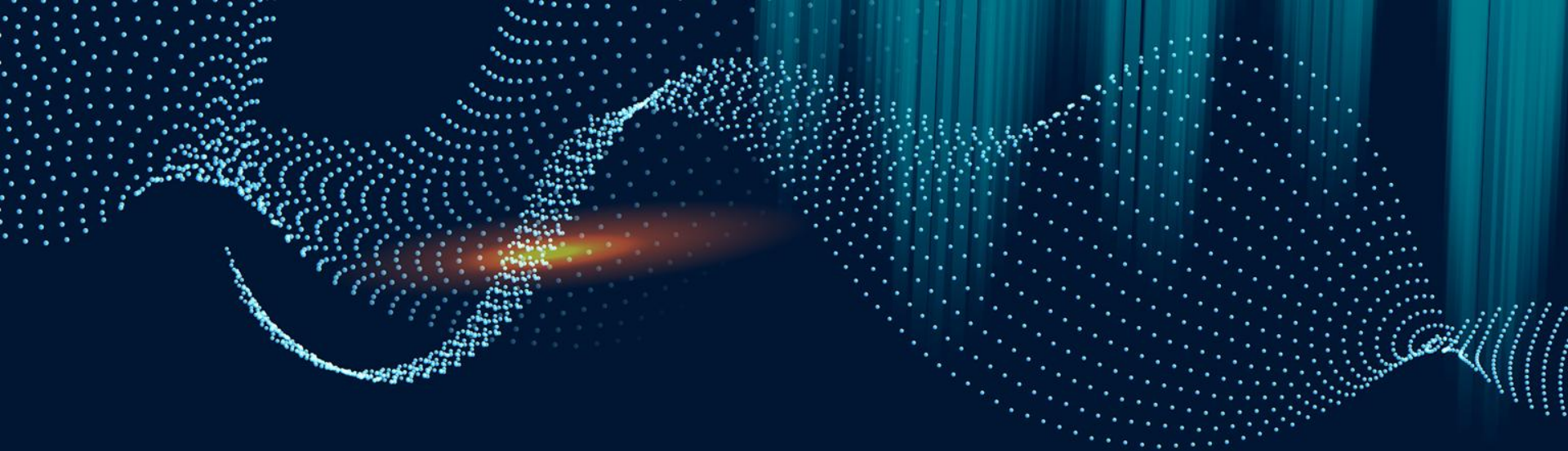
Learning by sending the error backward and fixing the weights.

Definition

- ❑ Backpropagation is the key algorithm to train neural networks.
- ❑ It calculates the error (difference between prediction and true output) and adjusts the weights to reduce that error.

Steps

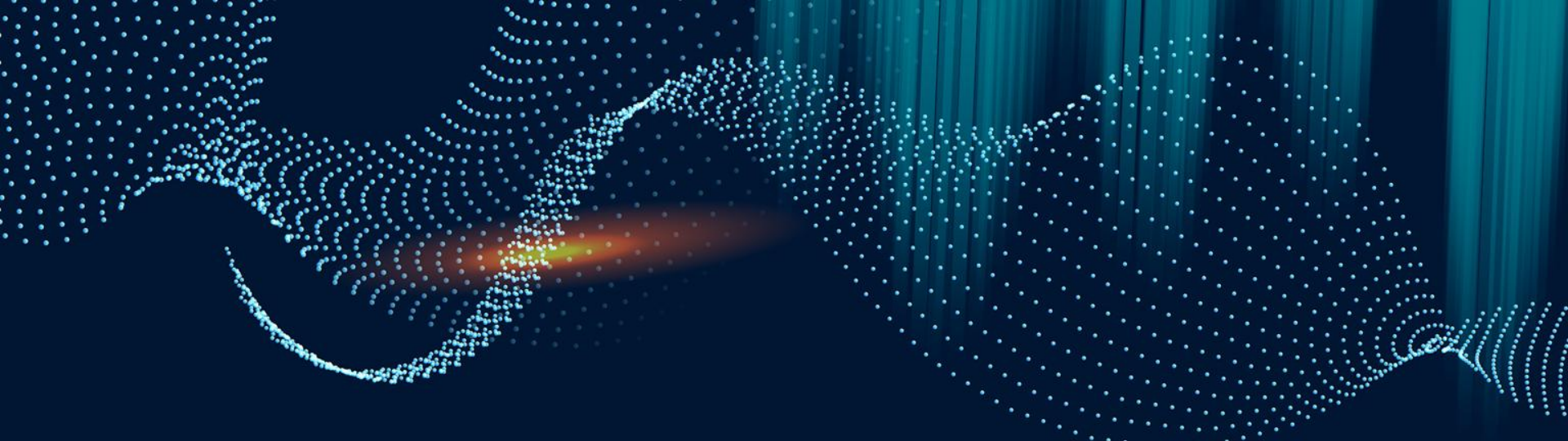
1. **Forward Pass:**
 - Pass input through the network and get output.
2. **Calculate Loss:**
 - Compare predicted output vs. true output.
3. **Backward Pass:**
 - Calculate how much each weight contributed to the error (using derivatives).
4. **Update Weights:**
 - Adjust weights slightly to reduce the error (using an optimizer like SGD or Adam).



Dropout

What is Dropout? Why use Dropout?

- ❑ Dropout is a regularization technique to prevent overfitting.
- ❑ During training, random neurons are "dropped" (turned off) with probability p .
- ❑ Remaining neurons are kept with probability $(1 - p)$.
- ❑ **Forces the network to not rely on specific neurons too much.**
- ❑ **Makes the model more general and better on unseen data.**
- ❑ **Acts like training many smaller networks at once.**
- ❑ **Dropout = Randomly turning off neurons during training to build stronger, more general models.**
- ❑ **Dropout layer is active during training. Dropout layer is turned off during testing/inference.**



Batch Size

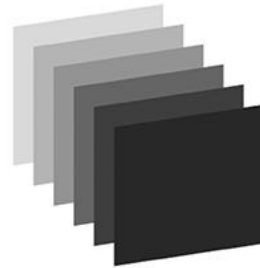
Definition

Number of training examples the model sees at one time before updating weights

- ❑ Training on the whole dataset at once (like 1 million images) would be too slow or use too much memory.
- ❑ So we split data into small batches (like 32, 64, 128 examples).
- ❑ **batch size of 32 is quite sufficient in order to take advantage of computational parallelism on GPUs.**
- ❑ **Model takes a small batch.**
- ❑ **Train loop: Forward pass → Loss calculation → Backward pass → Update weights.**
- ❑ **Repeat with next batch.**

Batch Size = How much data the model looks at before making a tiny improvement

$$\text{Iterations per Epoch} = \frac{\text{Total Samples}}{\text{Batch Size}}$$



Dataset of 6 images



A
Batch size: 3
Epoch: 1
Iteration: 2



B
Batch size: 2
Epoch: 1
Iteration: 3

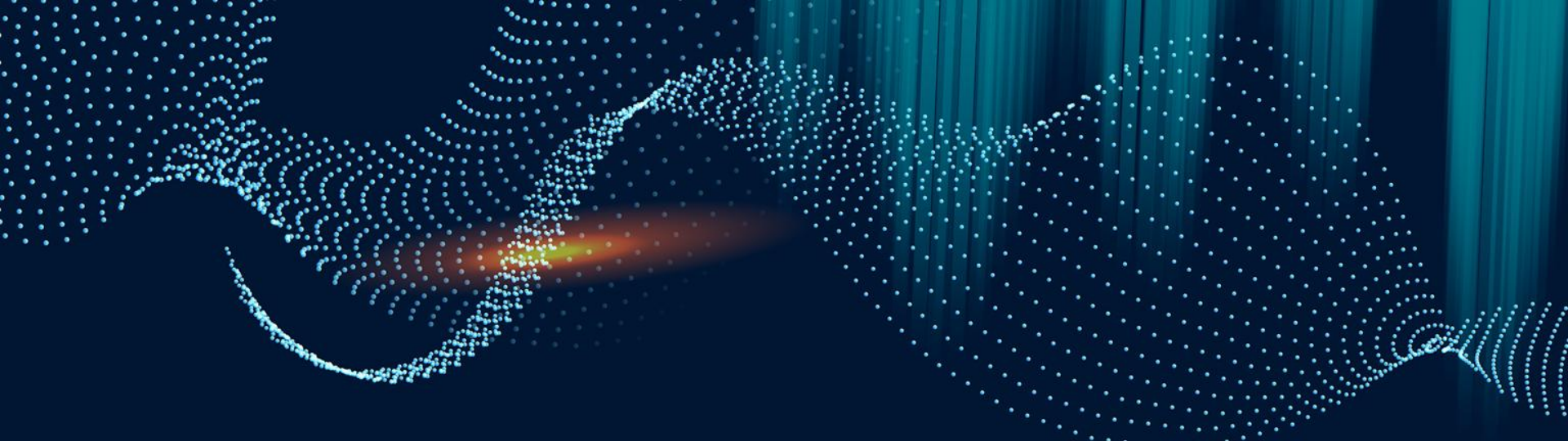


C
Batch size: 2
Epoch: 2
Iteration: 6

Exercise - 2

For each scenario below, fill in the missing values

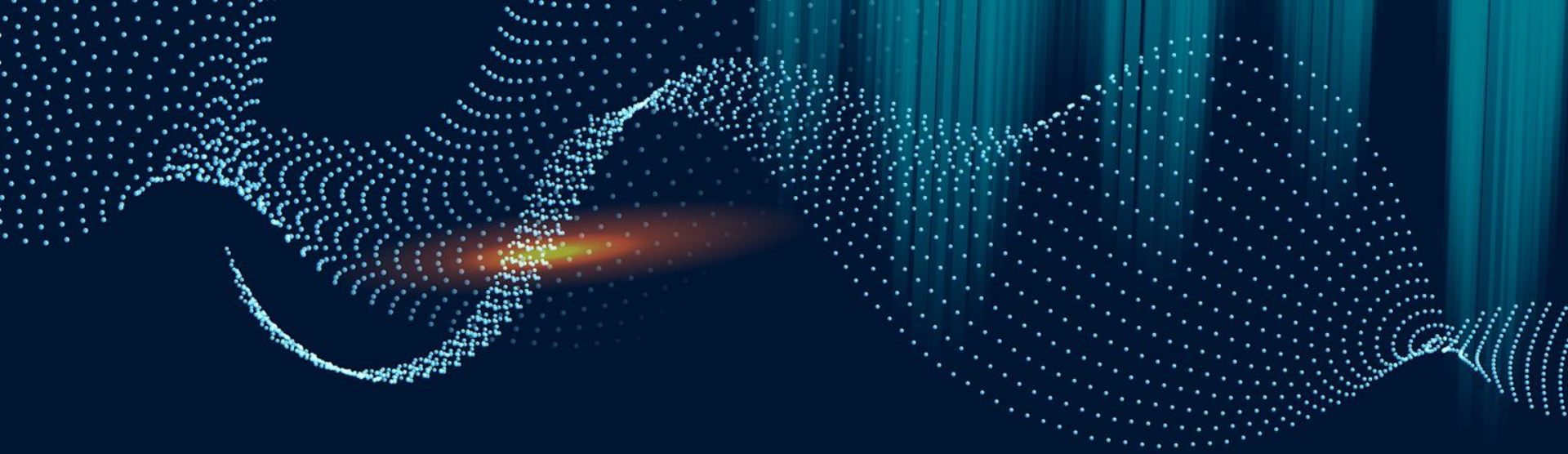
Total Samples	Batch Size	Epochs	Iterations per Epoch	Total Iterations
12	3	1	_____	_____
20	4	2	_____	_____
15	5	3	_____	_____
100	10	1	_____	_____
50	5	5	_____	_____



Batch Normalization

Data processing - Batch normalization

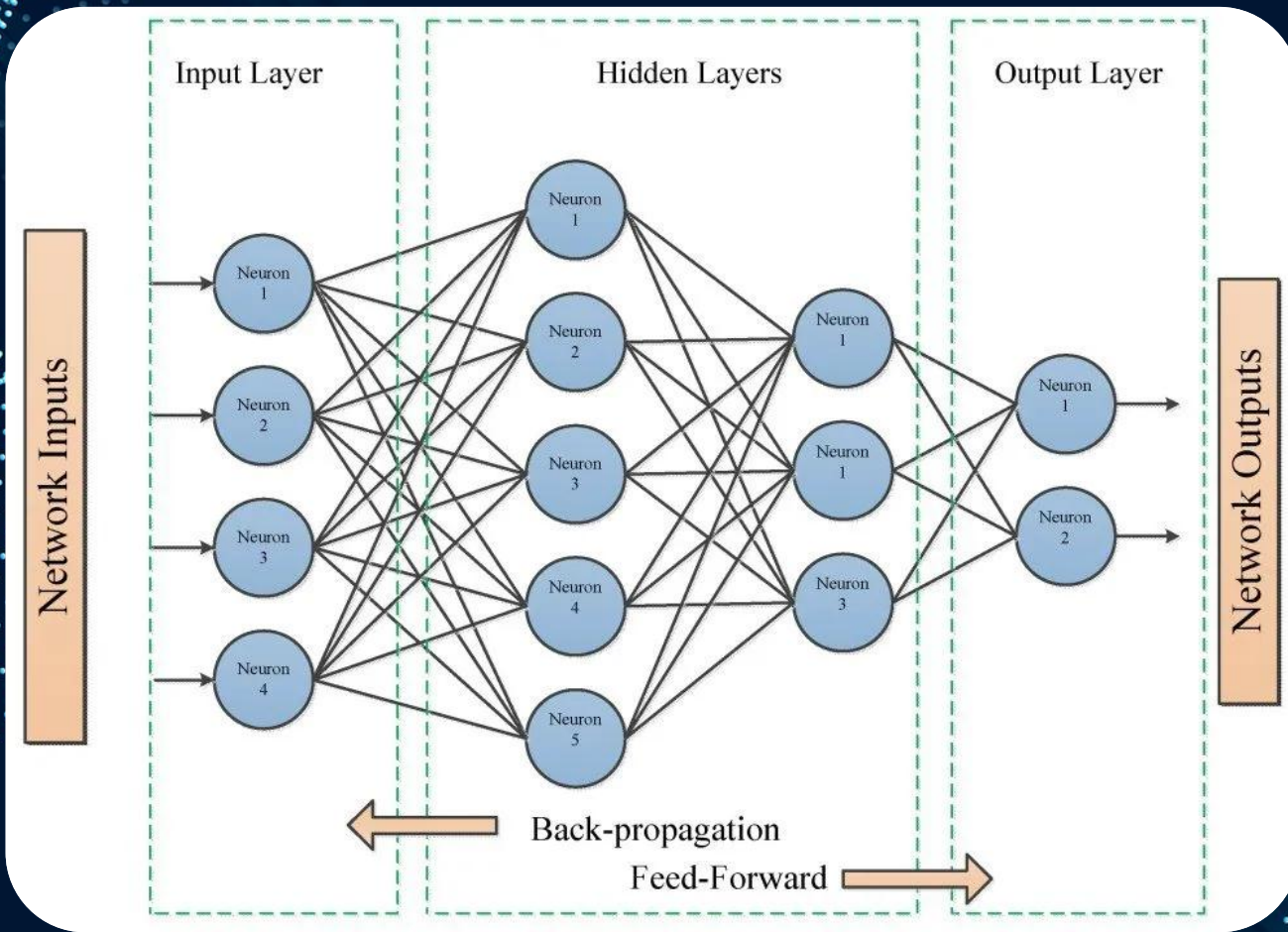
- ❑ A technique to normalize the inputs of each layer (mini-batch $\{x_i\}$) during training.
- ❑ Helps the network train faster and become more stable.
- ❑ After a Fully Connected or Convolutional Layer
- ❑ Before the Activation Function (like ReLU)
- ❑ Allows higher learning rates 🚀
- ❑ Reduces sensitivity to weight initialization 🎯
- ❑ Acts like regularization (reducing overfitting) 🔥

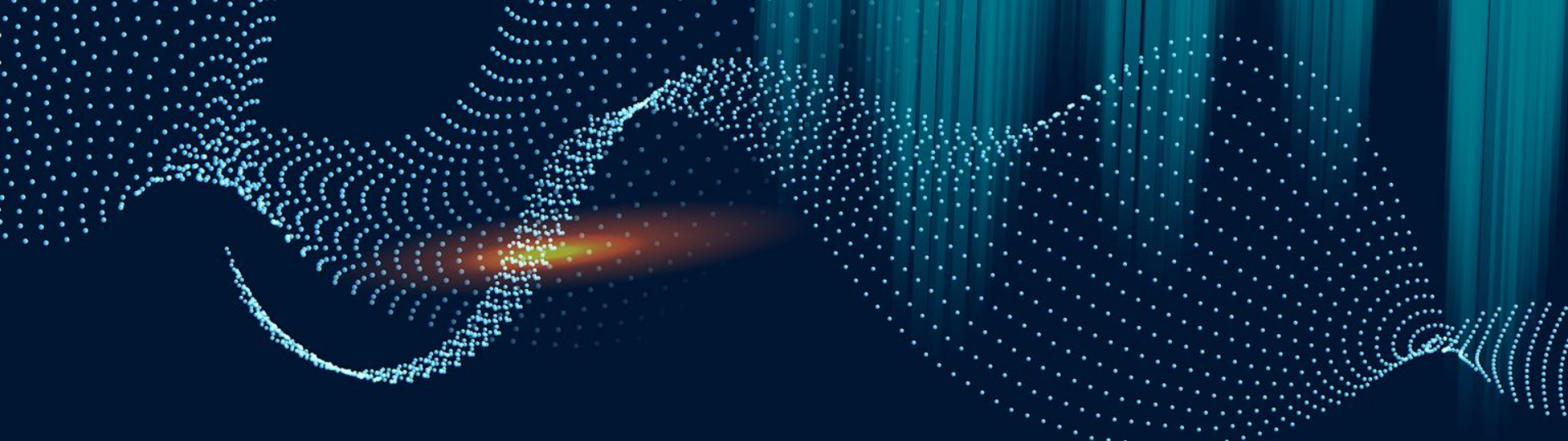


Dense Layer

Dense Layer (Fully Connected Layer)

- A Dense layer is the most basic layer in a neural network.
- Every neuron (node) is connected to every neuron in the previous layer.
- It takes inputs, multiplies them by weights, adds bias, and passes through an activation function.





Overfitting **vs** Underfitting

Overfitting

Learning: Model learns too much, even noise and random fluctuations in the training data.

Performance: Very good on training data, bad on new (test) data.

Cause: Model is too complex (too many layers/neurons).

Loss Behavior: Low training loss, high test loss.

Example: Memorizing answers without understanding.

Underfitting

Learning: Model learns too little, cannot even capture the patterns in training data.

Performance: Bad on both training and test data.

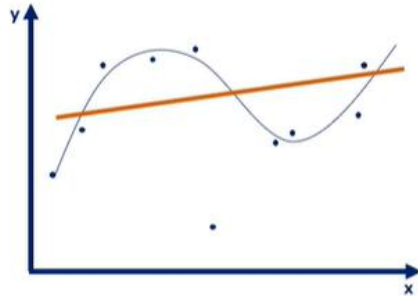
Cause: Model is too simple (not enough layers/neurons).

Loss Behavior: High training loss and high test loss.

Example: Guessing randomly without learning anything.

Underfitting and overfitting

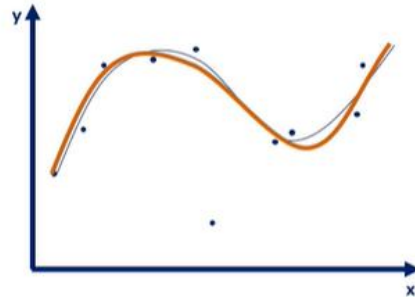
An **underfitted** model



Doesn't capture any logic

- High loss
- Low accuracy

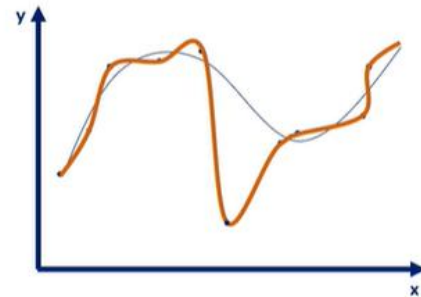
A **good** model



Captures the underlying logic of the dataset

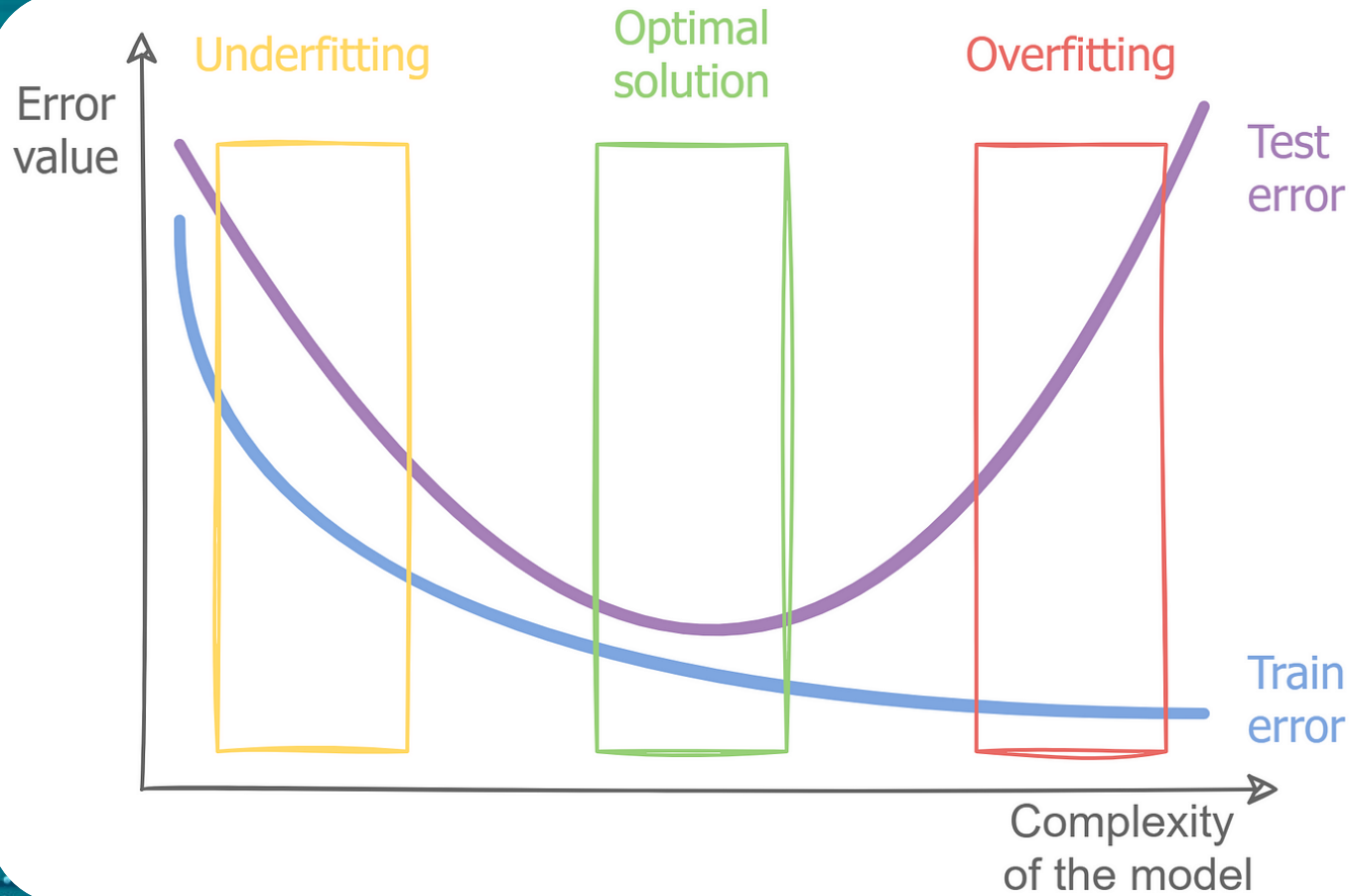
- Low loss
- High accuracy

An **overfitted** model



Captures all the noise, thus "missed the point"

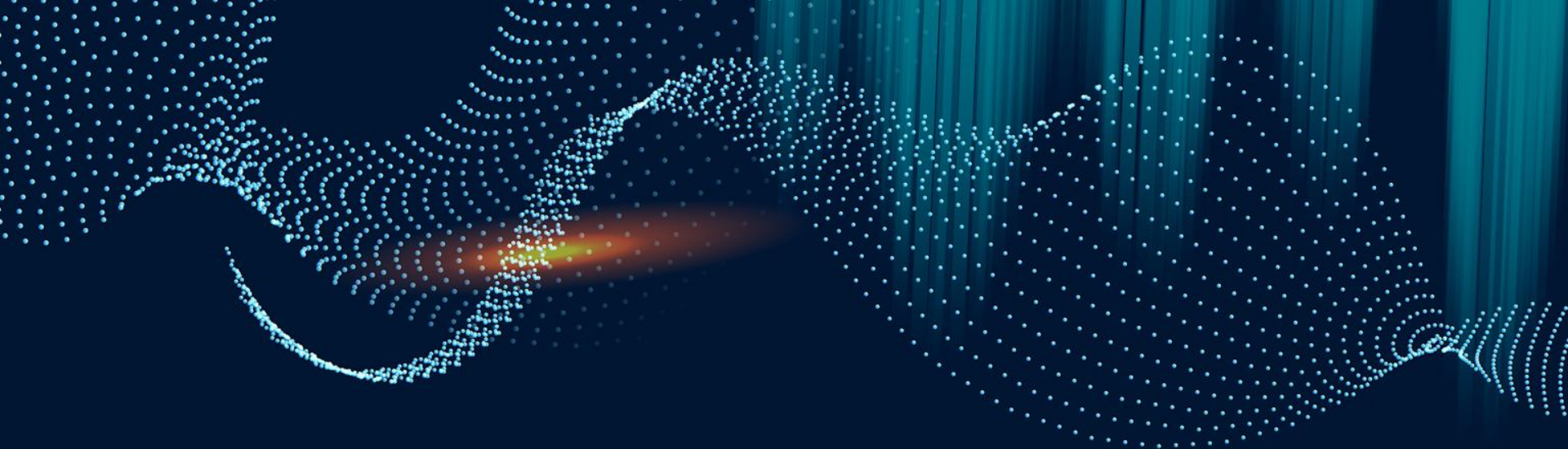
- Low loss
- Low accuracy



Solution

Techniques to fight underfitting and overfitting

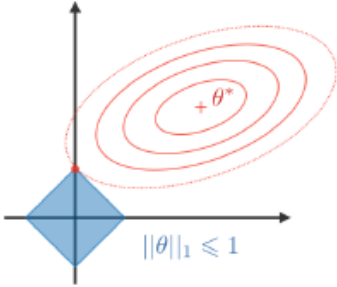
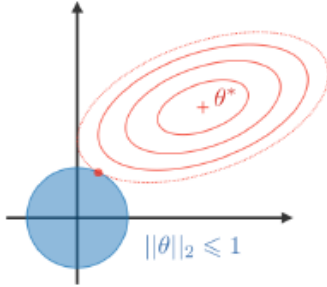
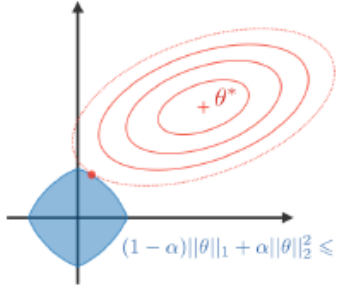
	Underfitting	Overfitting
Complexity of the model	More complex model Try a more powerful model with a larger number of parameters Ensemble learning More layers / number of neurons per layer	More simple model Try a less powerful model with a fewer number of parameters Less layers / number of neurons per layer
Regularization	Less regularization Decrease regularization	More regularization Increase regularization impact Early stopping, L1 / L2 regularization, dropout
Quantity of features	A larger quantity of features Get additional features, feature engineering, polynomial features, etc.	A smaller quantity of features Remove all additional features, feature selection
Data	Data cleaning, hold-out validation or cross validation. Getting more data most likely will not help	Data cleaning, hold-out validation or cross validation. Getting more data most likely will help (data augmentation)



Regularization

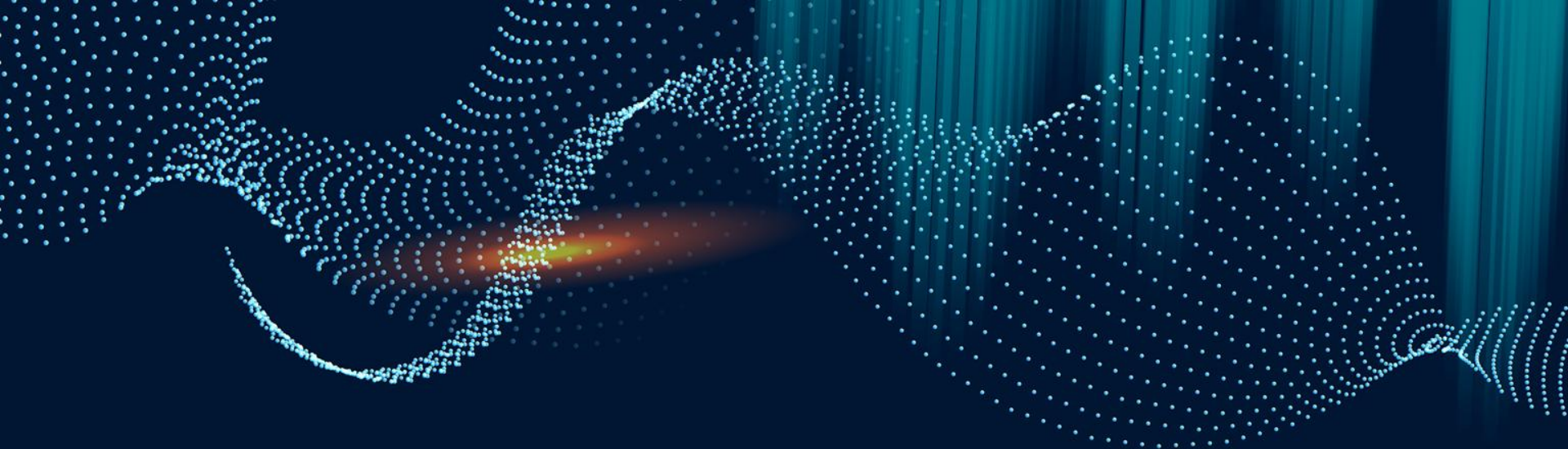
Why Regularization?

- Technique to prevent overfitting by adding a penalty to the loss function.
- Encourages simpler models that generalize better to new data.

LASSO	Ridge	Elastic Net
- Shrinks coefficients to 0 - Good for variable selection	Makes coefficients smaller	Tradeoff between variable selection and small coefficients
 $\theta _1 \leq 1$	 $\theta _2 \leq 1$	 $(1-\alpha) \theta _1 + \alpha \theta _2^2 \leq 1$
$\dots + \lambda \theta _1$ $\lambda \in \mathbb{R}$	$\dots + \lambda \theta _2^2$ $\lambda \in \mathbb{R}$	$\dots + \lambda \left[(1-\alpha) \theta _1 + \alpha \theta _2^2 \right]$ $\lambda \in \mathbb{R}, \alpha \in [0,1]$

Weight regularization

In order to make sure that the weights are not too large and that the model is not overfitting the training set, regularization techniques are usually performed on the model weights.



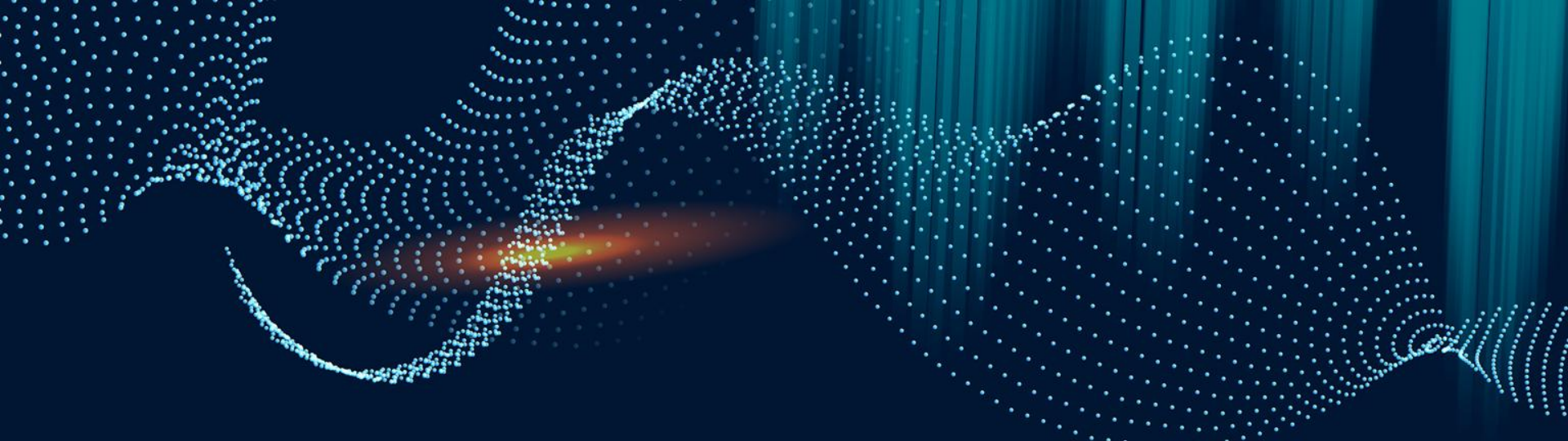
Data **Augmentation**

What is Data Augmentation?

- A technique to artificially expand the training dataset.
- Create new training examples by modifying existing ones.
- Helps the model generalize better and reduce overfitting.

Original	Flip	Rotation	Random crop
			
- Image without any modification	- Flipped with respect to an axis for which the meaning of the image is preserved	- Rotation with a slight angle - Simulates incorrect horizon calibration	- Random focus on one part of the image - Several random crops can be done in a row

Color shift	Noise addition	Information loss	Contrast change
			
- Nuances of RGB is slightly changed - Captures noise that can occur with light exposure	- Addition of noise - More tolerance to quality variation of inputs	- Parts of image ignored - Mimics potential loss of parts of image	- Luminosity changes - Controls difference in exposition due to time of day



Deep Learning Frameworks

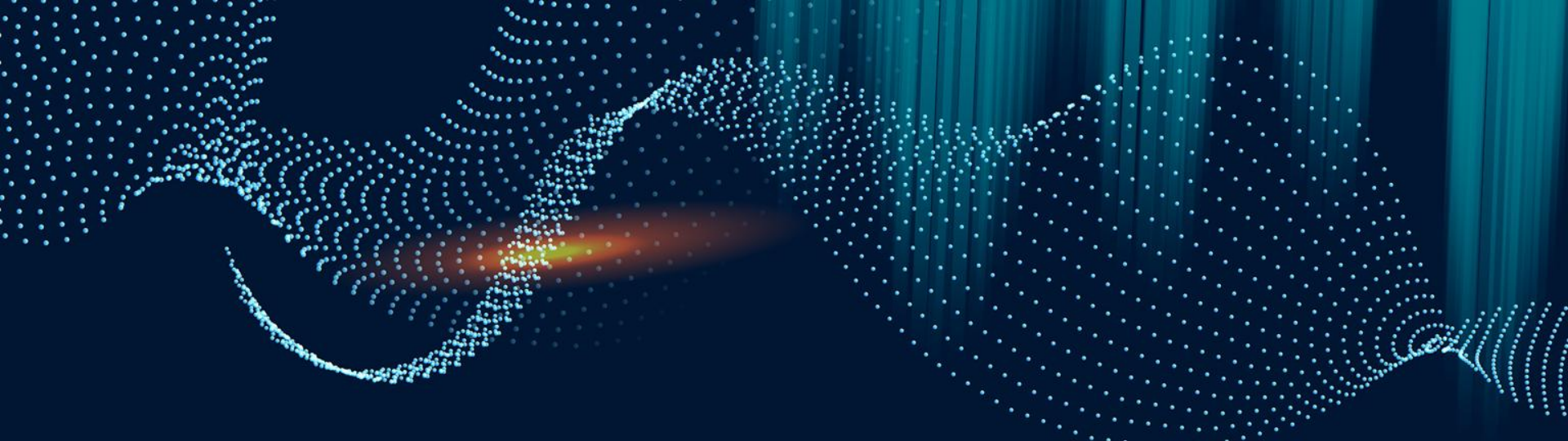
PyTorch & TensorFlow

PyTorch & TensorFlow

- ❑ Start with PyTorch for learning (easier coding + easier debugging).
- ❑ Use TensorFlow if you aim for large-scale deployment later.



Framework	Description	Key Strengths
PyTorch	Dynamic graph building (eager execution).	Easy to debug, flexible, loved by researchers.
TensorFlow	Static graph (default) + Eager mode available.	Production-ready, scalable, loved by industry.



Neural Network Playground



Epoch
000,000

Learning rate

1

Activation

Linear

Regularization

L1

Regularization rate

0.001

Problem type

Regression

DATA

Which dataset do you want to use?



Ratio of training to test data: 80%

Noise: 0

Batch size: 9

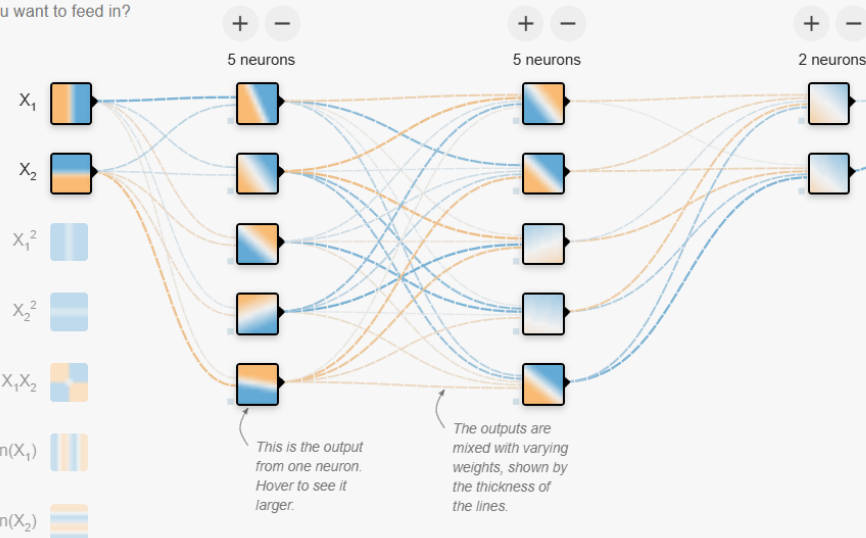
REGENERATE

FEATURES

Which properties do you want to feed in?

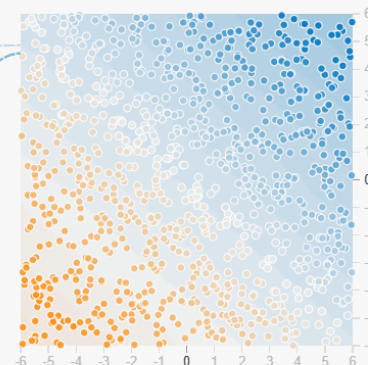
- X_1
- X_2
- X_1^2
- X_2^2
- X_1X_2
- $\sin(X_1)$
- $\sin(X_2)$

+ - 3 HIDDEN LAYERS

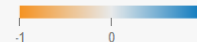


OUTPUT

Test loss 0.069
Training loss 0.070



Colors shows data, neuron and weight values.



☐ Show test data ☐ Discretize output

Presentation Topics

- ✓ Topic 1: Regularization Techniques
- ✓ Topic 2: Overfitting and Underfitting: Causes and Solutions
- 🕒 The presentation should be 5 to 10 minutes long, no more, and should use plots to explain.

Thank You

Do you have any questions?

pejman.ebrahimi@uni.li

